Experiment No. 9

Aim: Implementation of supervised learning algorithm like

1. Ada-Boosting
2. Random forests.

Theory:

Ada Boosting Algorithm:

AdaBoost (Adaptive Boosting) is a popular ensemble learning algorithm that combines multiple weak classifiers to form a strong classifier. The core idea behind AdaBoost is to sequentially train weak learners, typically decision trees with a single split (stumps), and place more emphasis on the data points that were misclassified by previous classifiers. In each iteration, AdaBoost adjusts the weights of the training samples, increasing the weights of the misclassified instances so that the subsequent weak learner focuses more on those difficult cases. This process continues for a specified number of rounds, or until the model achieves a desired level of accuracy.

The final model in AdaBoost is a weighted sum of the weak learners, where each learner's contribution is based on its accuracy. The more accurate a learner is, the higher its influence in the final decision. AdaBoost is particularly effective because it reduces both bias and variance, making it a versatile tool for a wide range of classification tasks. However, it is sensitive to noisy data and outliers since the algorithm tends to focus heavily on the hardest-to-classify examples, which can sometimes lead to overfitting. Despite this, AdaBoost remains a cornerstone in the ensemble learning paradigm, known for its simplicity and effectiveness in improving the performance of weak classifiers.

Random Forest Algorithm:

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning**,** which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.

As the name suggests, "Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing continuous variables, as in the case of regression, and categorical variables, as in the case of classification. It performs better for classification and regression

tasks. In this tutorial, we will understand the working of random forest and implement random forest on a classification task.

Input:

Let's understand using the Titanic Dataset as the Input for performing the Algorithms

Step 1: Importing Necessary Packages

```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.legend_handler import HandlerLine2D
```

Step 2: Read the Input Dataset

```
[2]: train = pd.read_csv("train.csv")
     print(train.shape)
```

```
(891, 12)
```

```
[3]: train.head(5)
```

[3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Step 3: Checking for Missing Values

```
[4]: NAs = pd.concat([train.isnull().sum()], axis=1, keys=["Train"])
     NAs[NAs.sum(axis=1) > 0]
```

[4]:

	Train
Age	177
Cabin	687
Embarked	2

Step 4: Filling the Missing Values

[5]:
```
# Filling missing Age values with mean
train["Age"] = train["Age"].fillna(train["Age"].mean())
```

[6]:
```
# filling missing Embarked values with mode
train["Embarked"] = train["Embarked"].fillna(train["Embarked"].mode()[0])
```

[7]:
```
# filling missing Cabin values with mode
train["Cabin"] = train["Cabin"].fillna(train["Cabin"].mode()[0])
```

[8]:
```
train["Pclass"] = train["Pclass"].apply(str)
```

[9]:
```
# Getting Dummies from all other categorical vars
for col in train.dtypes[train.dtypes == "object"].index:
    for_dummy = train.pop(col)
    train = pd.concat([train, pd.get_dummies(for_dummy, prefix=col)], axis=1)

bool_cols = train.select_dtypes(include='bool').columns
train[bool_cols] = train[bool_cols].astype(int)

train.head()
```

Step 5: Training and Testing

[11]:
```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(train, labels, test_size=0.25)
```

Step 6: Applying Random Forest Classifier

[12]:
```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()
rf.fit(x_train, y_train)
```

[12]:
```
RandomForestClassifier
RandomForestClassifier()
```

[13]:
```
y_pred = rf.predict(x_test)
```

[14]:
```
from sklearn.metrics import roc_curve, auc
false_positive_rate, true_positive_rate, thresholds = roc_curve(y_test, y_pred)
roc_auc = auc(false_positive_rate, true_positive_rate)
roc_auc
```

[14]:
```
0.7993150684931508
```

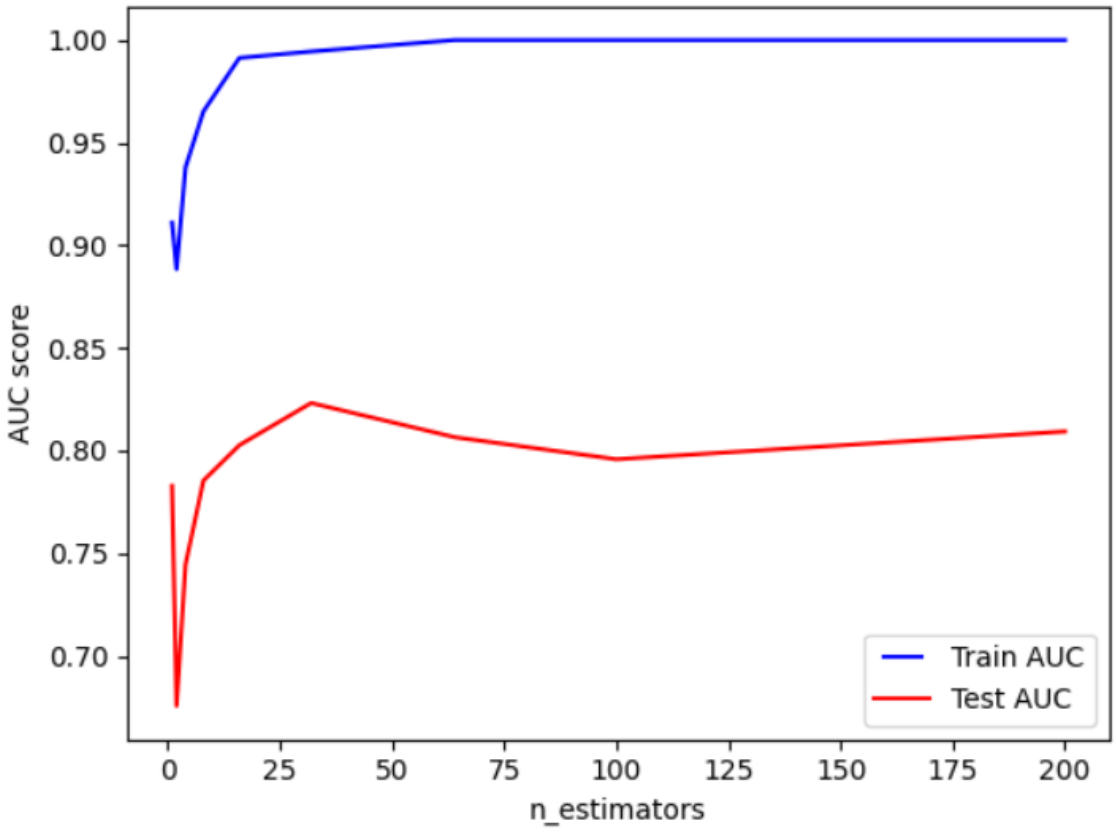
[15]:
```
n_estimators = [1, 2, 4, 8, 16, 32, 64, 100, 200]
train_results = []
test_results = []

for estimator in n_estimators:
    # Initialize the RandomForestClassifier
    rf = RandomForestClassifier(n_estimators=estimator, n_jobs=-1)
    rf.fit(x_train, y_train)

    # Predict on training data
    train_pred = rf.predict(x_train)
    false_positive_rate, true_positive_rate, thresholds = roc_curve(y_train, train_pred)
    roc_auc = auc(false_positive_rate, true_positive_rate)
    train_results.append(roc_auc)

    # Predict on test data
    y_pred = rf.predict(x_test)
    false_positive_rate, true_positive_rate, thresholds = roc_curve(y_test, y_pred)
    roc_auc = auc(false_positive_rate, true_positive_rate)
    test_results.append(roc_auc)

# Plotting the results
line1, = plt.plot(n_estimators, train_results, "b", label="Train AUC")
line2, = plt.plot(n_estimators, test_results, "r", label="Test AUC")
plt.legend(handler_map={line1: HandlerLine2D(numpoints=2)})
plt.ylabel("AUC score")
plt.xlabel("n_estimators")
plt.show()
```



Step 7: Applying AdaBoost Classifier

[16]:
```
from sklearn.ensemble import AdaBoostClassifier
ada_boost = AdaBoostClassifier(algorithm='SAMME')
ada_boost.fit(x_train, y_train)
```

[16]:
```
AdaBoostClassifier(algorithm='SAMME')
```

[17]:
```
y_pred = ada_boost.predict(x_test)
```

[18]:
```
from sklearn.metrics import roc_curve, auc
false_positive_rate, true_positive_rate, thresholds = roc_curve(y_test, y_pred)
roc_auc = auc(false_positive_rate, true_positive_rate)
roc_auc
```

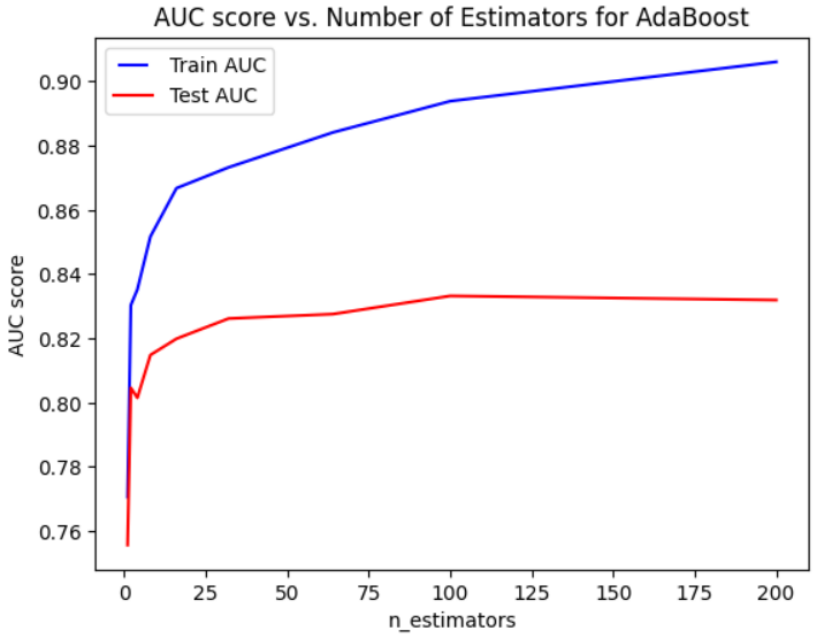
[18]:
```
0.7931963470319634
```

[19]:
```
n_estimators = [1, 2, 4, 8, 16, 32, 64, 100, 200]
train_results = []
test_results = []

for estimator in n_estimators:
    # Initialize the AdaBoostClassifier with SAMME algorithm
    ada_boost = AdaBoostClassifier(n_estimators=estimator, algorithm='SAMME')
    ada_boost.fit(x_train, y_train)

    # Predict on training data
    train_pred = ada_boost.predict_proba(x_train)[:, 1]  # Probability estimates for ROC curve
    false_positive_rate, true_positive_rate, _ = roc_curve(y_train, train_pred)
    roc_auc = auc(false_positive_rate, true_positive_rate)
    train_results.append(roc_auc)

    # Predict on test data
    y_pred = ada_boost.predict_proba(x_test)[:, 1]  # Probability estimates for ROC curve
    false_positive_rate, true_positive_rate, _ = roc_curve(y_test, y_pred)
    roc_auc = auc(false_positive_rate, true_positive_rate)
    test_results.append(roc_auc)
```



<u>Conclusion</u>: Thus, we successfully studied and implemented supervised learning algorithms like Random Forest and Ada Boosting